

BuildPipeline Refactoring Kata – Resumo da Refatoração

Este documento resume a refatoração realizada no projeto BuildPipeline Refactoring Kata. O objetivo foi melhorar a clareza, organização e testabilidade do código responsável por executar testes, realizar deploy e enviar emails com o resultado do build.

Funcionalidade do Sistema

A classe Pipeline executa três tarefas principais: rodar testes do projeto, realizar deploy caso os testes passem e enviar um email com o resultado do processo.

Principais Problemas no Código Original

Problema
Método run() muito longo e com várias responsabilidades
Uso de strings mágicas espalhadas pelo código
Lógica duplicada para verificar resultados
Condicionais aninhadas difíceis de entender
Ausência de testes automatizados

Refatorações Aplicadas

Durante a refatoração foram realizadas várias melhorias importantes: criação de testes unitários com JUnit e Mockito, extração de métodos menores para separar responsabilidades, criação de constantes para substituir strings mágicas e simplificação da lógica de envio de email.

Testes Automatizados

Foram adicionados testes cobrindo os principais cenários: execução bem-sucedida, falha nos testes, falha no deploy e verificação de logs e envio de emails.

Benefícios da Refatoração

Aspecto	Antes	Depois
Legibilidade	Método grande	Métodos menores e claros
Testes	Nenhum teste	Cobertura com JUnit
Manutenção	Strings espalhadas	Constantes centralizadas
Complexidade	Condicionais aninhadas	Lógica mais simples

Executando os Testes

Comandos básicos para rodar os testes do projeto:

```
cd java  
./gradlew test
```

Documento resumido a partir do BuildPipeline Refactoring Kata.